

Chapter 4. Errors and Debugging

In Node, error objects are important. They are used to communicate the existence of problems and provide context and details about them. Good handling of errors makes programs more reliable and secure.

This chapter is all about errors and how to deal with them. We'll first talk about throwing and catching errors and then discuss the different types of errors and what layered error management looks like. We'll then explore debugging Node code and discuss some preventive measures that we can use to reduce potential errors.

Throwing and Catching Errors

An error in Node is an object instantiated from the `Error` class. Sometimes you'll need to create your own error objects, like this:

```
const error = new Error('Some message');
```

This is a generic error object, which is not something you should ever use, as there is a better option that we'll discuss shortly. Node also has built-in errors, which are basically error objects already created for you. The next section covers those.

What exactly do you do with an error object? Well, regardless of the type of error and whether you created it or it was built-in, whenever you're in a place in the code where a condition should not happen, you need to signal the details of that problem to whoever is using your code. You can do that by *throwing* the error.

For example, let's say we want to create a function to calculate the square root of a number. One restriction such a function should have is to block any operation that tries to use the function with a negative number, since there is no square root for a negative number.

That case is a user input error that's expected, and if it happens, the function should provide a signal that it's being used incorrectly. One simple way to do that is to throw an error:

```
function calculateSquareRoot(number) {  
  if (number < 0) {  
    throw new Error(  
      'Cannot calculate square root of negative number'  
    );  
  }  
  
  return Math.sqrt(number);  
}  
  
calculateSquareRoot(-1);  
// Error: Cannot calculate square root of negative number
```

Throwing an error will make the current operation stop completely. When you throw any errors in Node, the process will crash and exit. Here's what happens when we execute this code:

```
$ node index.js  
file:///Path/To/File:3  
    throw new Error(  
          ^  
Error: Cannot calculate square root of negative number  
    at calculateSquareRoot (file:///Path/To/File:3:11)  
    at file:///Path/To/File:11:1
```

This might feel wrong. Why crash an entire application because of a simple wrong usage of a single function?

The simple answer here is that it's a problem that should not be ignored and should be handled somehow. Ignoring errors might lead to unin-

tended consequences and compromises the integrity of the application. A function that's called with the wrong input might have incorrect data propagated throughout the application. That's really not good. In some cases, it might even cause a security vulnerability. In this simple example, an unvalidated input might be the path for an exploitative attack on the application.

For errors related to the application environment, not handling them might lead to leaks and potential system resources exhaustion. It's just too much of a risk to ignore any errors. In your core modules, you should always throw errors when you're in a condition that means a problem. It's your signal that a problem should either be acknowledged and handled properly or make the application crash.

Now, the modules that use your core modules might make an exception for certain errors. For example, a module that uses your `calculateSquareRoot` function might decide to implement an exception for having the function called with a negative number. That can be done by *catching* the error.

Whenever you write code that uses other code (like calling a function), you need to always remember that this other code might throw errors. As the user of that code, you can decide what to do with these errors. You do that by catching the error and handling it somehow.

For example, say that you decided that when `calculateSquareRoot` gets called with a negative number, it should just print a warning, not exit the whole application. Here's how you do that:

```
try {  
  let result = calculateSquareRoot(-1);  
} catch (error) {  
  console.error(error.message);  
}
```

Because the call is wrapped in a `try...catch`, and the catch block just outputs the error to the console, using this code and calling

`calculateSquareRoot` with a negative number will not crash the process. This error is now a handled exception.

The problem is, the handled exception here is not just the error for a negative number argument. It's any error within the `calculateSquareRoot` function. If the function throws any other error, they will all be caught by the `try...catch` and ignored on this level as well. We should make exceptions for only the errors that we know of. Any unknown errors should still be thrown.

To do that, we need a way to conditionally handle the error. We need to check the *type* of an error. Let's talk about error types next.

Types of Errors

Different types of errors usually have different handling strategies. Knowing these types and their underlying reasons for existence helps identify what's going on when you see them.

The most important types of errors in Node are *standard errors*, *system errors*, and *custom errors*. Let's explore them with some examples.

Standard Errors

Standard errors are the built-in errors that are provided by JavaScript itself. These errors are thrown by the JavaScript engine when it encounters any unexpected condition that prevents the normal flow of a program. For example, when wrong syntax is used, or a variable is referenced before it was declared, or a function is used in an invalid way.

The main standard error classes in JavaScript are `SyntaxError`, `ReferenceError`, `RangeError`, and `TypeError`. All of these are subclasses of the `Error` class.

SyntaxError

This is the type of error thrown when we try to run code that's not valid JavaScript. We get them if we use a reserved keyword, declare a variable with an invalid name, misplace a comma, bracket, or parenthesis, or do anything else that does not follow the syntactic rules of the language:

```
> console.log('Hello world';  
console.log('Hello world';  
          ^^^^^^^^^^^^^^^  
  
// Uncaught SyntaxError: missing ) after argument list  
>
```

ReferenceError

This is the type of error thrown when you try to use a variable that has not been declared or is not in the current scope:

```
console.log(x);  
// Uncaught ReferenceError: x is not defined  
  
let x = 5;
```

RangeError

This is the type of error thrown when you try to use a value that's not within the allowed range of values enforced by either an implementation limit or a memory limit—for example, if you use a method with an argument that's not within the accepted range:

```
(123.456).toFixed(101);  
// Uncaught RangeError: toFixed() digits argument  
//                               must be between 0 and 100
```

Calling a function recursively without an exit condition could also produce this error:

```
function f() {  
  f();  
}  
  
f(); // Uncaught RangeError: Maximum call stack size exceeded
```

TypeError

This is the type of error thrown when you try to use a value of an unexpected type anywhere a type is expected. For example, the method `thing.toUpperCase()` expects `thing` to be a string, but if you try to use that method with a number, you get a `TypeError`:

```
let thing = 42;  
console.log(thing.toUpperCase());  
  
// Uncaught TypeError: thing.toUpperCase is not a function
```

Other examples of `TypeError` include trying to modify a `const` variable, accessing properties on the `null` and `undefined` objects, and using the `new` keyword with nonconstructor functions, among a few more cases.

NOTE

These are the most common standard errors. There are a few more like `URIError`, which is related to handling URIs, and `EvalError`, which exists only for compatibility and is no longer thrown by modern JavaScript.

System Errors

System errors in Node are thrown when something unexpected happens on the system level. They are basically the problems that come up because of the environment and OS where a Node application is run—for example, if you try to read or write to a file that does not exist, use a network port that's already in use, send data over a closed socket, and so on.

These errors have specific codes that you can look up to understand why they happen. Here are a few common system error codes:

`ENOENT`

Thrown when accessing a file or directory that does not exist.

`EACCES`

Thrown when an operation does not have the right permissions. For example, if we try to write to a read-only file.

`EADDRINUSE`

Thrown when an HTTP (or TCP) server fails to start because the address it's trying to use is already in use.

`ECONNREFUSED`

Thrown when a connection is made to a server that's not running.

Other common system errors include `ETIMEDOUT` (operation timed out), `ECONNRESET` (connection reset by peer), `ENOTFOUND` (entity not found), `EPERM` (operation not permitted), and many more. If the error code looks like `ESOMETHING`, it's usually a system error code. You can see the full list of system errors using `os.constants.errno`, as shown in [Figure 4-1](#).

```
TERMINAL
node + - [ ] [ ] ...
○ $ node
>
> os.constants.errno
[Object: null prototype] {
  E2BIG: 7,
  EACCES: 13,
  EADDRINUSE: 48,
  EADDRNOTAVAIL: 49,
  EAFNOSUPPORT: 47,
  EAGAIN: 35,
  EALREADY: 37,
  EBADF: 9,
  EBADMSG: 94,
  EBUSY: 16,
  ECANCELED: 89,
  ECHILD: 10,
  ECONNABORTED: 53,
  ECONNREFUSED: 61,
  ECONNRESET: 54,
  EDEADLK: 11,
  EDESTADDRREQ: 39,
  ENOM... 33
```

Figure 4-1. System errors

TIP

These system error codes are POSIX codes. POSIX (Portable Operating System Interface) is a set of standards (based on the Unix OS) that are used for maintaining compatibility between different OSs.

System errors are part of the more general category of *operational errors*. Operational errors are the nonfatal errors that can be detected, expected, and—in most cases—handled gracefully within the code. They include system errors and other runtime errors not related to the OS, service errors related to external services (like a database), input errors (for unexpected external input), and timeout errors (when an operation takes too long), among a few others.

The next error category (custom errors) can also be considered a subset of operational errors.

Custom Errors

Custom errors in Node are the errors created by you, the developer, to handle specific conditions or scenarios in your code. The error we threw for the square root of a negative number is a generic custom error.

TIP

Custom errors are also known as *user-specified errors* (*user* in this case refers to the developer, not the end user of the application).

You'll be working with custom errors more often than other types. We create them for specific error handling and to add additional context or properties to the errors relevant to our application's logic. For example, a custom `UserNotFoundError` might be used when something tries to access the record of a user not in the system, or a `TransactionFailedError` might be thrown when a transaction fails to complete successfully. It's not uncommon for a Node application to have tens, if not hundreds, of custom errors.

While a generic instance of the `Error` class is a custom error, you should give your custom errors their own classifications. We can do that by extending the `Error` class, and then instantiating objects out of that extended class:

```
class ValidationError extends Error {
  constructor(message, fieldName) {
    super(message);
    this.name = 'ValidationError';
    this.fieldName = fieldName;
  }
}

// To use:
// throw new ValidationError('Some message', 'some_field');
```

This custom `ValidationError` class can then be used to throw its specific error when you are validating something. For example, if you need to make sure that a `user` object has a `username` field, you can throw this error if it does not:

```
function validateUser(user) {
  if (!user.username) {
    throw new ValidationError('Username is required', 'username');
  }
}
```

```
}  
}
```

Note how the error is specific and helpful; when it gets thrown, we'll know exactly why it did, and we can get the helpful details that's packaged in it. Here's another example:

```
try {  
  validateUser({}); // An empty object is not a valid user object  
} catch (error) {  
  if (error instanceof ValidationError) {  
    console.error(  
      `Error in field '${error.fieldName}': ${error.message}`,  
    );  
  } else {  
    console.error(`Unexpected error: ${error.message}`);  
    throw error;  
  }  
}
```

TIP

Keep the extending of the `Error` class to one level, and don't create a tree of errors. If your code is split into different structures (*domain*, *api*, *app*, etc.), you can categorize your errors in different modules under these structures.

No matter how you organize your custom errors, be as detailed as you can when naming them! Is there a condition where a connection to a database can't be established? Throw a `DatabaseConnectionError`. Is there a condition where a user record could not be found using a unique ID? Throw a `CouldNotFindUserFromIdError`. Note how I used noun-based and verb-based names in these two examples. Pick your preference.

Note how in this example, after making the exception for the known and expected `ValidationError`, I am *rethrowing* any other errors. This is very important, so let's talk about that in the next section.

ASSERTION ERRORS

Another category of errors in Node is assertion errors. These are specific errors used primarily in testing and development environments. They are usually used when the code expects something to be true but finds it false.

Assertion errors are also commonly used for writing tests and when debugging and validating assumptions during development. If an assertion starts to fail anywhere in the code, that means there's likely a bug or a misunderstanding leading up to that failure point, which needs to be addressed before releasing any code changes.

Assertion errors in Node are thrown using the `assert` module, which provides a simple set of assertion tests like the following:

```
assert(value[, message])
```

Is the value true?

```
assert.equal(actual, expected[, message])
```

Does the `actual` value equal the `expected` value?

```
assert.doesNotMatch(string, regexp[, message])
```

Does the `string` value match the `regexp` pattern?

When an assertion method is called, it evaluates its specific condition. If the condition is not met, the assertion method throws an `AssertionError`, which details the nature of the failure. If the condition is met, the program continues execution as normal.

There are many other assertion methods for various conditions. See the Node [documentation page for the `assert` module](#). We'll see examples of how to use this module in [Chapter 8](#).

Layered Error Management

Say you have modules `A`, `B`, and `C`, where module `B` uses module `A`, and module `C` uses module `B`. When thinking about error management, you need to remember this kind of layered structure.

With the understanding of error types and how to create custom error objects, we can now throw distinct errors for distinct conditions and give the upper layers that are using our core modules the chance to make exceptions for these distinct errors. However, after making these exceptions, a catch block should always end in a `throw error` call to rethrow any unknown errors:

```
try {
  // Some code that might throw multiple types of errors
} catch (error) {
  if (error instanceof KnownErrorType) {
    // Handle specific known error
  } else if (error instanceof AnotherKnownErrorType) {
    // Handle another specific known error
  } else {
    // Log and rethrow unknown errors
    console.log('Encountered an unexpected error: ', error);
    throw error;
  }
}
```

Without rethrowing the error object (which is unknown after making exceptions for the known ones), we would be silently ignoring it and making the application run in an unstable or incorrect state without any indication (to all upper layers) that something has gone wrong in this layer. That also makes debugging problems a lot harder since the context of the error would be lost.

By rethrowing the error, you propagate it to the higher layers. Not rethrowing the error is basically hiding it from higher layers.

There is another way to manage errors. Instead of throwing them, we can pass them forward to other parts of the application that use our core modules and let these parts either handle them or forward them again.

The error forwarding method is commonly used in asynchronous and event-driven programming, where an error encountered in one function is not immediately handled within that function but is instead sent along with any data to the next function in the sequence.

The error-first callback style discussed in [Chapter 3](#) is basically one way to do error forwarding. When an error occurs within a function, the error is packaged up and passed as the first argument to the next callback. This allows the next function in line to check for the presence of an error and decide how to handle it:

```
function fetchData(callback) {
  getSomeDataFromAnAPI((err, data) => {
    if (err) {
      callback(err); // Forwarding the error to the callback
      return;
    }
    callback(null, data); // No error occurred, continue as normal
  });
}

fetchData((err, data) => {
  if (err) {
    console.log('Error fetching data: ', err);
    return;
  }
  console.log('Data received: ', data);
});
```

Promise-based functions are another way to forward errors. An error within the promise implementation is forwarded using the `reject` method:

```
function fetchData() {
  return new Promise((resolve, reject) => {
```

```

    getSomeDataFromAnAPI((err, data) => {
      if (err) {
        reject(err); // Forwarding the error through rejection
      } else {
        resolve(data); // Resolving the promise if no error
      }
    });
  });
}

fetchData()
  .then((data) => {
    console.log('Data received: ', data);
  })
  .catch((err) => {
    console.log('Error fetching data: ', err);
  });

```

However, we don't need promises or callbacks to implement error forwarding. We can do it simply by making functions return either data (success) or error (failure), or even both (partial success). This can be done either by unifying the return into an object with both an `error` property and `data` property, or by using a simpler approach of returning either a `data` object or an `error` object. The latter is a bit tricky as every function would need to return two different types. I would recommend the latter approach only if the Node project is using TypeScript, which adds static typing to JavaScript. We'll explore the basics of TypeScript for Node in [Chapter 10](#).

Here's an example of a function unifying the return into an object:

```

function fetchData() {
  try {
    let data = getSomeData();
    return { error: null, data: data };
  } catch (error) {
    return { error: error, data: null };
  }
}

```

```
const result = fetchData();
if (result.error) {
  console.log('Error fetching data:', result.error);
} else {
  console.log(result.data);
}
```

Error forwarding can be used to maintain a clean and predictable flow of both data and errors through the application's operations. However, every part of the application needs to either handle errors or forward them, just as how, without error forwarding, every part of the application needs to either handle errors or throw them. Error management has to be a style that's enforced throughout the entire application.

Debugging in Node

When a code problem needs to be investigated, there are multiple ways to do the debugging in Node. The simplest way is to log information around the problem to understand what's going on. To start, simple `console.log` statements can be used to find out where exactly the problem is happening (in case the error stack was not helpful):

```
console.log('Starting application...');
app.init();
console.log('Application initialized successfully.');
```

If you don't see the second log message, that means `app.init()` has a problem.

Further logging can be used to print out data, verify expectations, and track how things change. It's a simple yet powerful way to quickly identify issues in a development environment.

For more featured debugging, Node has a built-in command-line debugging utility that can be started using the `inspect` argument:

```
$ node inspect script.js
```

This command starts your Node application with an inspector attached, allowing you to pause execution, step through the code, and inspect variables at runtime. You can set breakpoints directly from your code by adding the `debugger;` statement, which will cause the execution to pause whenever that point is reached.

The built-in debugger is certainly useful, but it's limited. There is a more powerful and featured option. You can use the Chrome browser DevTools to debug Node applications just as easily as debugging web applications. All you have to do is start the Node process with the `--inspect` flag:

```
$ node --inspect script.js
```

You can also use the `--inspect-brk` flag to break at the start of the debugged script.

After that, you can connect to the running Node process using Chrome's *chrome://inspect* page (see [Figure 4-2](#)). The running Node process will be listed there, and when you click it, you'll have all the power of DevTools to use on your Node code: the debugger, the performance profile, the memory inspector, and the real-time console.

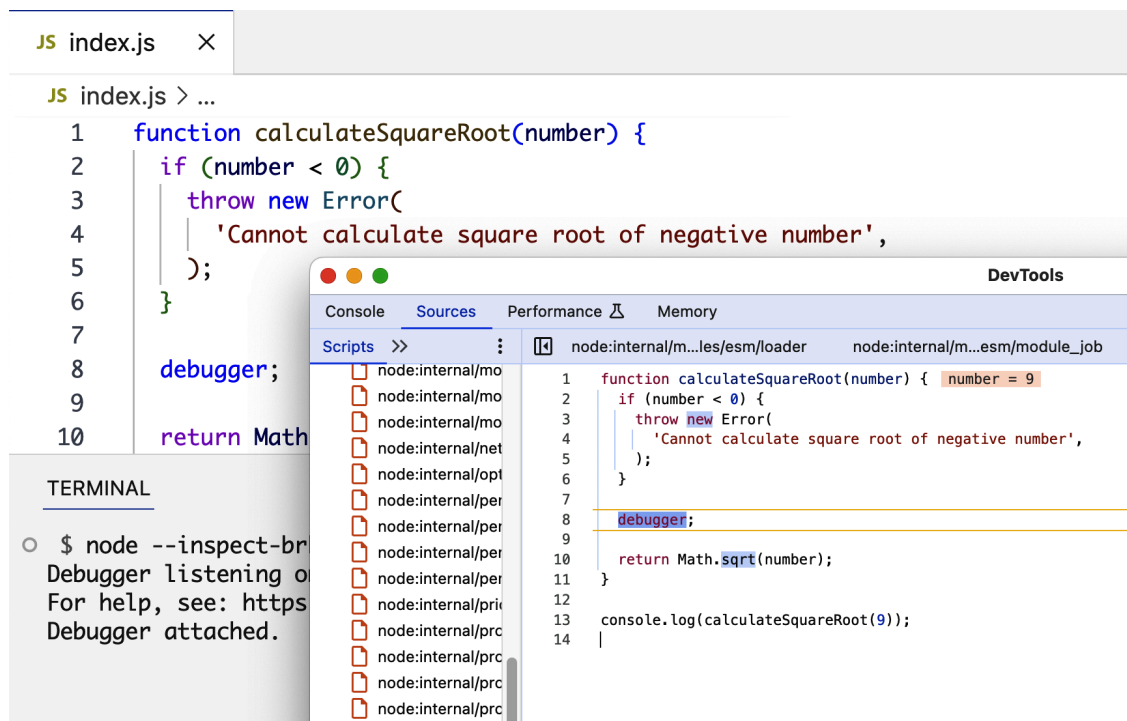


Figure 4-2. Using Chrome DevTools with Node

Node also has a built-in performance profiler that you can run with the `-prof` flag. It samples the stack at regular intervals during program execution and records the results of these samples, along with important optimization events.

Additionally, Node has multiple tracing flags that can be used to print additional information to error stacks. You can see them along with what they do in the `node --help` output:

<code>\$ node --help grep trace</code>	
<code>--enable-source-maps</code>	Source Map V3 support for stack traces
<code>--trace-deprecation</code>	show stack traces on deprecations
<code>--trace-event-categories=...</code>	comma separated list of trace event
<code>--trace-event-file-pattern=...</code>	for the trace-events data, it supports
<code>--trace-exit</code>	show stack trace when an environment
<code>--trace-promises</code>	show stack traces on promise
<code>--trace-sigint</code>	enable printing JavaScript stacktrace
<code>--trace-sync-io</code>	show stack trace when use of sync 10 is
<code>--trace-tls</code>	prints TLS packet trace information to
<code>--trace-uncaught</code>	show stack traces for the <code>`throw`</code>
<code>--trace-warnings</code>	show stack traces on process warnings

TIP

Another benefit to using clear and descriptive names for objects in your code (especially functions) is that it helps with debugging. Well-named objects in stack traces and error logs make it easier to identify where and why an issue might be happening.

Preventive Measures

While dealing with errors in general is unavoidable, there are some tools and practices you can adopt to reduce the chances of having to deal with unknown errors.

Code Quality Tools

When you write a function that takes inputs, the first thing you should do is make sure that the inputs are what you expect them to be. Are they the right type? Are they in the correct format? Are they in the right range? Throw errors if they are not.

Using TypeScript with Node might at first feel like making things a lot more complicated, but in reality, the benefits you get are far greater than the little troubles.

TypeScript will point out problems in your code before the code is even run. You passed the wrong argument type to a function? You made a typo accessing a property on an object? You called a method that does not exist? TypeScript will point out all these problems, and many more, right away. TypeScript can also help you write error management code in a much better way.

The ESLint linter can also help find problems in your code before the code is run. You can use it to identify patterns that lead to bugs, and it has many other features for enforcing code style and best practices. ESLint also has a lot of powerful plugins that can extend its scope and make it work with many libraries and frameworks.

We'll expand more on both TypeScript and ESLint in [Chapter 10](#).

Immutable Objects

A truly immutable object cannot be changed once it's created. Any updates should create new objects instead. Using immutable data structures (from a library like `Immutable.js`, for example) helps you avoid a ton of potential problems that come from changing data unintentionally. Even simple immutability concepts in JavaScript itself, like using `const` instead of `let`, freezing objects, and using read-only properties, are all very helpful methods to avoid many problems.

Testing

All code has to be tested one way or another. We manually test the code all the time, but we can't manually test all the cases after every change. A small change anywhere might cause bugs in places you would never expect. That's why automating tests by writing code that tests your code is really important.

Writing tests today in Node is easier than ever. Node has built-in modules for testing and assertions. The `node:test` module provides a way to organize your tests and describe them. The `node:assert` module provides assertion methods to implement the logic of the tests and throw assertion errors when the expectations are not met. Testing code in Node is the topic of [Chapter 8](#).

Code Reviews

This goes without saying, but a solid process of reviewing all code in a project goes a long way toward finding problems early and finding better ways of doing things. I'd say every line of code should be reviewed by at least two developers.

Summary

Many problems can happen during the execution of a Node program. Some problems are out of our control, whereas others can and should be handled. Problems in Node are presented with error objects that are either built-in or customized by the developers. Core modules throw these error objects, and modules that use core modules can catch error objects and handle them if they choose to.

When an error happens, a Node process will crash and exit. This is a good thing because you don't want a program to continue running in an unknown state. However, if an error is expected, the program does not need to crash. An exception is basically an error for which we made an exception to not stop the entire program.

There are several types of errors in Node. We have standard errors from JavaScript, system errors, custom errors, and assertion errors.

In a layered application structure, errors need to propagate between the layers. That can be done using the `throw` method, or with a more structured forwarding of error objects.

Simple log messages can be used to do simple debugging in Node. Node has a built-in command-line debugging utility, and Chrome DevTools can also be used for more featured debugging of Node applications.

There are a few tools and practices that can be used to reduce the chances of having to deal with errors. Tools like TypeScript and ESLint are very powerful in that domain. Validating inputs, writing tests, using immutable data structures, and performing code reviews are all good practices to consider for that purpose too.

In the next chapter, let's talk about package management in Node and learn more about the features of Node's package manager.

